

# MQTT for IoT Systems Practice

# Setting Up Mosquitto Broker

Install and run Mosquitto:

```
# Ubuntu/Debian
sudo apt install mosquitto mosquitto-clients

# Start the broker
sudo systemctl start mosquitto

# Test: subscribe to a topic
mosquitto_sub -h localhost -t "sensors/#" -v

# Test: publish a message (in another terminal)
mosquitto_pub -h localhost -t "sensors/room-A/temp" -m '{"temp": 72.4}'
```

# ESP32 Publishing via MQTT

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

WiFiClient  espClient;
PubSubClient mqtt(espClient);

void setup() {
  WiFi.begin("MySSID", "MyPassword");
  mqtt.setServer("192.168.1.100", 1883); // broker IP
}

void loop() {
  if (!mqtt.connected()) mqtt.connect("esp32-001");

  // Build JSON payload
  StaticJsonDocument<128> doc;
  doc["device_id"]   = "esp32-001";
  doc["temperature"] = readTemperature();
  doc["humidity"]    = readHumidity();

  char payload[128];
  serializeJson(doc, payload);

  // Publish to topic
  mqtt.publish("sensors/room-A/temperature", payload);
  delay(1000); // send every second – very lightweight!
}
```

# Python Server Subscribing to MQTT

The backend subscribes to sensor topics and saves to database:

```
import paho.mqtt.client as mqtt
import json
from datetime import datetime

def on_connect(client, userdata, flags, rc):
    print("Connected to broker")
    client.subscribe("sensors/#") # subscribe to ALL sensor topics

def on_message(client, userdata, msg):
    data = json.loads(msg.payload)
    topic = msg.topic # e.g. "sensors/room-A/temperature"

    record = {
        "topic": topic,
        "device_id": data.get("device_id"),
        "temperature": data.get("temperature"),
        "humidity": data.get("humidity"),
        "received_at": datetime.utcnow().isoformat()
    }
    db.save(record) # save to database
    print(f"Saved: {record}")

client = mqtt.Client()
client.on_connect = on_connect
```

# Two-Way Communication — Actuator Control

MQTT enables **commands back to devices** (bidirectional):

Dashboard → Broker → ESP32

Topic: `actuators/room-A/hvac`

Payload: `{"action": "set_temp", "setpoint": 70}`

```

// ESP32: subscribe to command topic
void setup() {
    mqtt.setServer("192.168.1.100", 1883);
    mqtt.setCallback(onCommand);
}

void onCommand(char* topic, byte* payload, unsigned int length) {
    String msg = String((char*)payload).substring(0, length);
    StaticJsonDocument<128> doc;
    deserializeJson(doc, msg);

    if (String(topic) == "actuators/room-A/hvac") {
        float setpoint = doc["setpoint"];
        setHVACTemperature(setpoint);    // control the actuator
    }
}

void loop() {
    if (!mqtt.connected()) {
        mqtt.connect("esp32-001");
        mqtt.subscribe("actuators/room-A/#");    // subscribe to commands
    }
    mqtt.loop();    // process incoming messages
}

```